

Practical 1:

Aim: Creating Simple Minimal API Endpoints:

1. Create MapGet endpoint
2. Return JSON data
3. Add MapPost endpoint
4. Test using browser/Postman

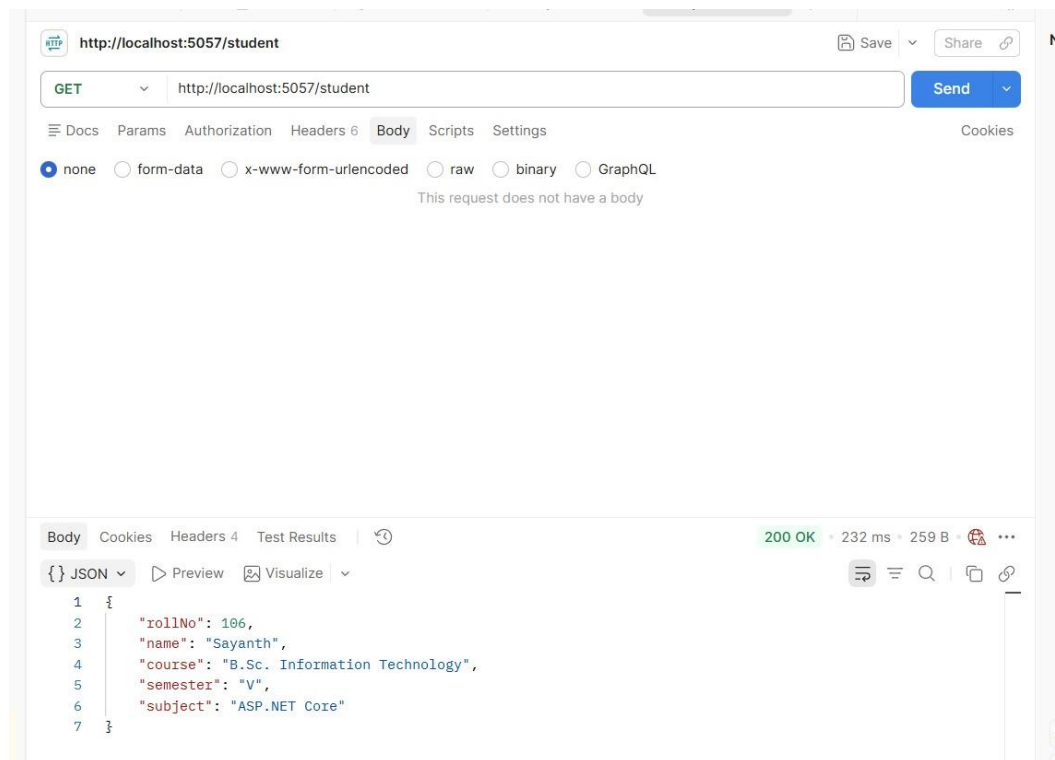
Codes:

Program.cs code:

```
var builder = WebApplication.CreateBuilder(args); var app = builder.Build();  
app.MapGet("/", () =>  
{  
    return "Welcome to ASP.NET Core Minimal API Practical";  
});  
app.MapGet("/student", () =>  
{ var student = new {  
    RollNo = 97,  
    Name = "Vidhi Salian",  
    Course = "B.Sc. Information Technology",  
    Semester = "V",  
    Subject = "ASP.NET Core" }; return  
student;  
});  
app.MapPost("/student", (Student student) =>  
{ return new {  
  
    };  
});
```

```
Message = "Student data received successfully", StudentName = student.Name,  
Course = student.Course app.Run(); class Student  
  
{ public string Name { get; set; } public string Course { get;  
set; }  
}
```

Output:



Practical 2:

Aim: Creating Controller-Based API

1. Create Product model
2. Implement GET and POST methods
3. Return proper HTTP status codes

Codes: Models/Product.cs

code:

```
namespace Practical2_ControllerAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
}
```

Controllers/ProductsController code:

```
using Microsoft.AspNetCore.Mvc; using
Practical2_ControllerAPI.Models; namespace
Practical2_ControllerAPI.Controllers
{
    [ApiController] [Route("api/[controller]")] public
    class ProductsController : ControllerBase
```

```

{ static List<Product> products = new List<Product>
{
new Product { Id = 1, Name = "Laptop", Price = 55000 }, new Product { Id = 2, Name =
"Mouse", Price = 500 }, new Product { Id = 3, Name = "Keyboard", Price = 1200 }
};

[HttpGet] public IActionResult
GetProducts()

{ return Ok(products); } [HttpGet("{id}")] public
ActionResult GetProductById(int id)
{
var product = products.FirstOrDefault(p => p.Id == id); if
(product == null)
{ return NotFound("Product not found"); } return
Ok(product);
}

[HttpPost] public IActionResult
AddProduct(Product product)
{ if (product == null)
{ return BadRequest("Invalid product data"); } products.Add(product); return
Created("api/products/" + product.Id, product);
} } }

```

Program.cs code:

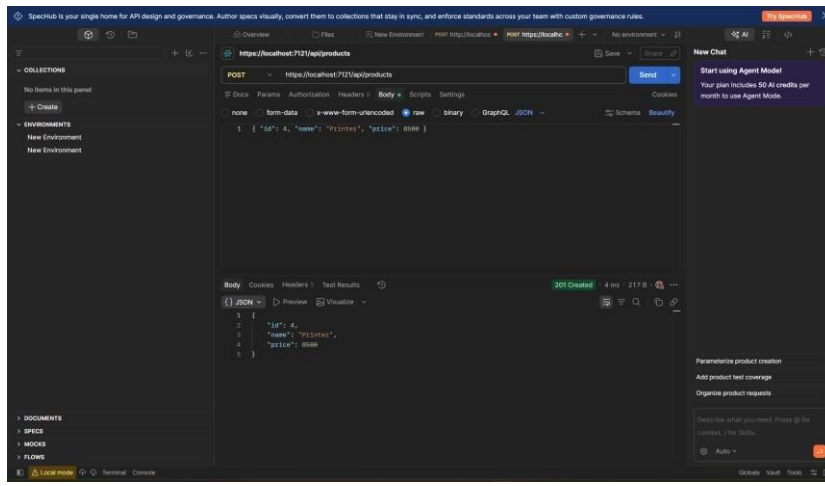
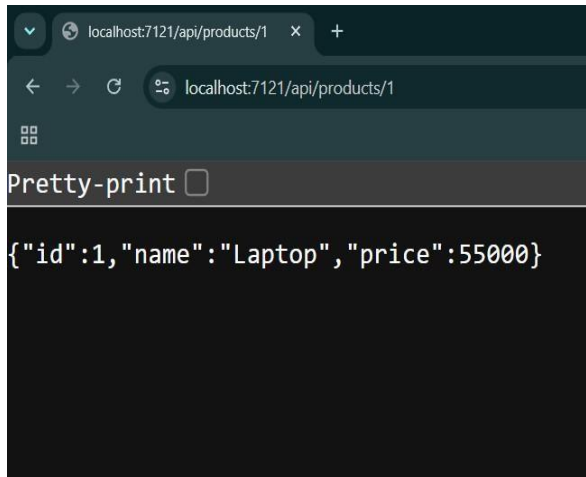
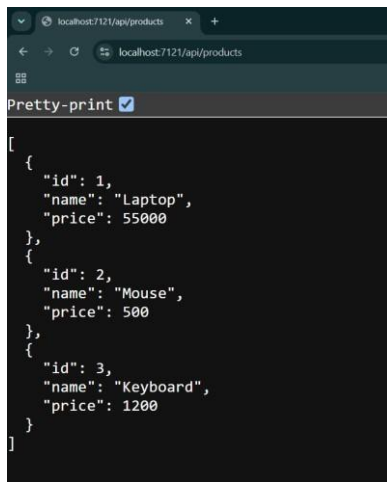
```

var builder = WebApplication.CreateBuilder(args); builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer(); builder.Services.AddSwaggerGen();

```

```
var app = builder.Build(); if
(app.Environment.IsDevelopment())
{ app.MapOpenApi(); } app.UseHttpsRedirection(); app.UseAuthorization();
app.MapControllers(); app.Run();
```

Output:



Practical 3:

Aim: Implementing Full CRUD Operations 1.

Add PUT and DELETE methods

2. Use in-memory list as data store
3. Test all endpoints using Postman

Codes:

Models/Product.cs code:

```
namespace Practical3_CRUDAPI.Models
{
    public class Product {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
}
```

Controllers/ProductsController code:

```
using Microsoft.AspNetCore.Mvc; using
Practical3_CRUDAPI.Models; namespace
Practical3_CRUDAPI.Controllers { [ApiController]
[Route("api/[controller]")] public class
ProductsController : ControllerBase { static
List<Product> products = new List<Product> {
new Product { Id = 1, Name = "Laptop", Price = 55000 }, new Product { Id = 2, Name =
"Mouse", Price = 500 }, new Product { Id = 3, Name = "Keyboard", Price = 1200 } };
[HttpGet] public IActionResult
```

```

GetAllProducts()
{ return Ok(products); } [HttpGet("{id}")] public
ActionResult GetProductById(int id) { var product =
products.FirstOrDefault(p => p.Id == id); if
(product == null)
{ return NotFound("Product not found"); } return
Ok(product); } [HttpPost] public ActionResult
AddProduct(Product product) {
products.Add(product); return Created("api/products/"
+ product.Id, product); } [HttpPut("{id}")] public ActionResult
UpdateProduct(int id, Product updatedProduct) { var product =
products.FirstOrDefault(p => p.Id == id); if (product
== null)
{ return NotFound("Product not found"); } product.Name =
updatedProduct.Name; product.Price = updatedProduct.Price; return
Ok(product); } [HttpDelete("{id}")] public ActionResult
DeleteProduct(int id) { var product = products.FirstOrDefault(p =>
p.Id == id); if
(product == null)
{ return NotFound("Product not found"); } products.Remove(product); return
Ok("Product deleted successfully"); } } }

```

Program.cs code:

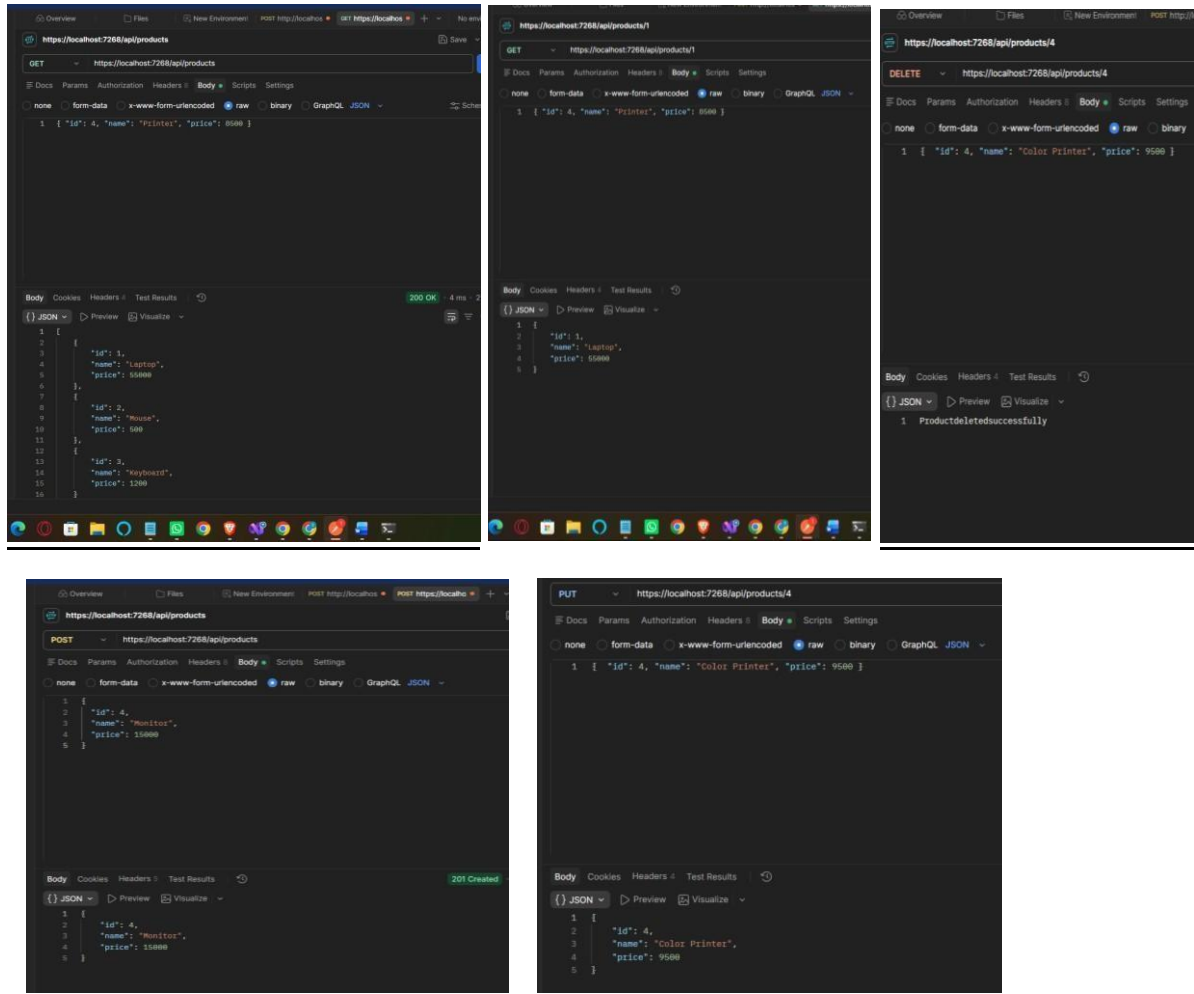
```

var builder = WebApplication.CreateBuilder(args); builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer(); builder.Services.AddSwaggerGen();

```

```
var app = builder.Build(); if (app.Environment.IsDevelopment()) { app.UseSwagger();  
app.UseSwaggerUI(); } app.UseHttpsRedirection(); app.UseAuthorization();  
app.MapControllers(); app.Run();
```

Output:



Practical 4:

Aim: Implementing Basic Middleware

1. Create simple request logging middleware
2. Register middleware in Program.cs
3. Observe execution flow

Codes:

Program.cs code(1):

```
var builder = WebApplication.CreateBuilder(args); var app = builder.Build(); app.Use(async
(context, next) =>
{
    Console.WriteLine(" ----- ");
    Console.WriteLine("Request Method : " + context.Request.Method);
    Console.WriteLine("Request Path : " + context.Request.Path); await next();
    Console.WriteLine("Response Status Code : " + context.Response.StatusCode);
    Console.WriteLine(" ----- "); } ); app.Run();
```

Program.cs code(2):

```
var builder = WebApplication.CreateBuilder(args); var app = builder.Build(); app.Use(async
(context, next) =>
{ Console.WriteLine("Middleware 1 Start"); await next();
```

```
Console.WriteLine("Middleware 1 End"); }); app.Use(async
(context, next) =>

{ Console.WriteLine("Middleware 2 Start"); await next();
Console.WriteLine("Middleware 2 End"); }); app.Run();
```

Output:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\shashan\source\repos\AspNetCore\AspNetCore
Middleware
-----
Request Method : GET
Request Path : /
Response Status Code : 200
-----
|
```

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\shashan\source\repos\AspNetCore\AspNetCore
Middleware
Middleware 1 Start
Middleware 2 Start
Middleware 2 End
Middleware 1 End
|
```

Practical 5:

Aim: Implementing Dependency Injection

1. Create service interface and implementation
2. Register service (scoped)
3. Inject service into controller

Codes:

Services/IMessageService.cs code:

```
namespace Practical5_DependencyInjection.Services
{ public interface IMessageService { string GetMessage(); } }
```

Services/MessageService.cs code:

```
namespace Practical5_DependencyInjection.Services
{ public class MessageService : IMessageService
{ public string GetMessage()
{ return "Welcome to ASP.NET Core Dependency Injection"; } } }
```

Program.cs code:

```
using Practical5_DependencyInjection.Services; var builder =
WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IMessageService, MessageService>();
```

```
builder.Services.AddControllers();

builder.Services.AddOpenApi(); var app = builder.Build(); //

Configure the HTTP request pipeline. if
(app.Environment.IsDevelopment()) { app.MapOpenApi(); }

app.UseHttpsRedirection();      app.UseAuthorization();

app.MapControllers(); app.Run();
```

Controllers/MessageController code:

```
using Microsoft.AspNetCore.Mvc; using
Practical5_DependencyInjection.Services; namespace
Practical5_DependencyInjection.Controllers {
[ApiController] [Route("api/[controller]")] public class
MessageController : ControllerBase { private readonly
IMessageService _messageService; public
MessageController(IMessageService messageService)
{ _messageService = messageService; }
[HttpGet] public IActionResult
GetMessage()
{ return Ok(_messageService.GetMessage()); } } }
```

Output:

OverviewFilesNew EnvironmentPOST http://localhosGET http://localhost.

HTTP

http://localhost:5198/api/message

SaveShare

GET

http://localhost:5198/api/message

Send

DocsParamsAuthorizationHeaders 6BodyScriptsSettings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

This request does not have a body

BodyCookiesHeaders 4Test Results

200 OK71 ms194 B

RawPreviewVisualize

1Welcome to ASP.NET Core Dependency Injection Sayanth

Practical 6

Aim: Implementing Dependency Injection

1. Apply attribute routing
2. Use route parameters
3. Implement simple route constraint

Codes:

Models/ Student.cs code:

```
namespace Practical6_Routing.Models
{
    public class Student {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Marks { get; set; }
    }
}
```

Controllers/ StudentsController.cs code:

```
using Microsoft.AspNetCore.Mvc;
using Practical6_Routing.Models;
namespace Practical6_Routing.Controllers {
    [ApiController]
    [Route("api/[controller]")]
    public class StudentsController : ControllerBase
    {
        private static List<Student> students = new List<Student>
        {
            new Student { Id = 1, Name = "Priya", Marks = 85 },
            new Student { Id = 2, Name = "Rahul", Marks = 82 },
            new Student { Id = 3, Name = "Amit", Marks = 78 }
        };

        [HttpGet]
        public IActionResult GetAllStudents() {
            return Ok(students);
        }

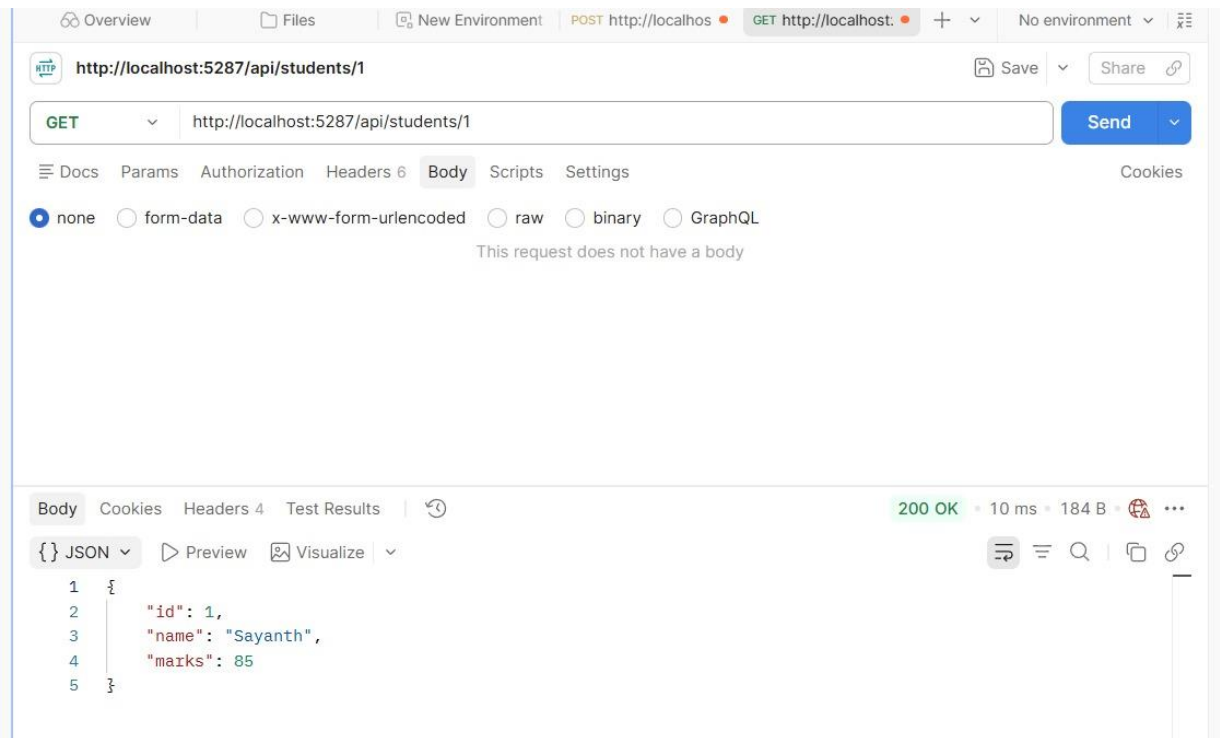
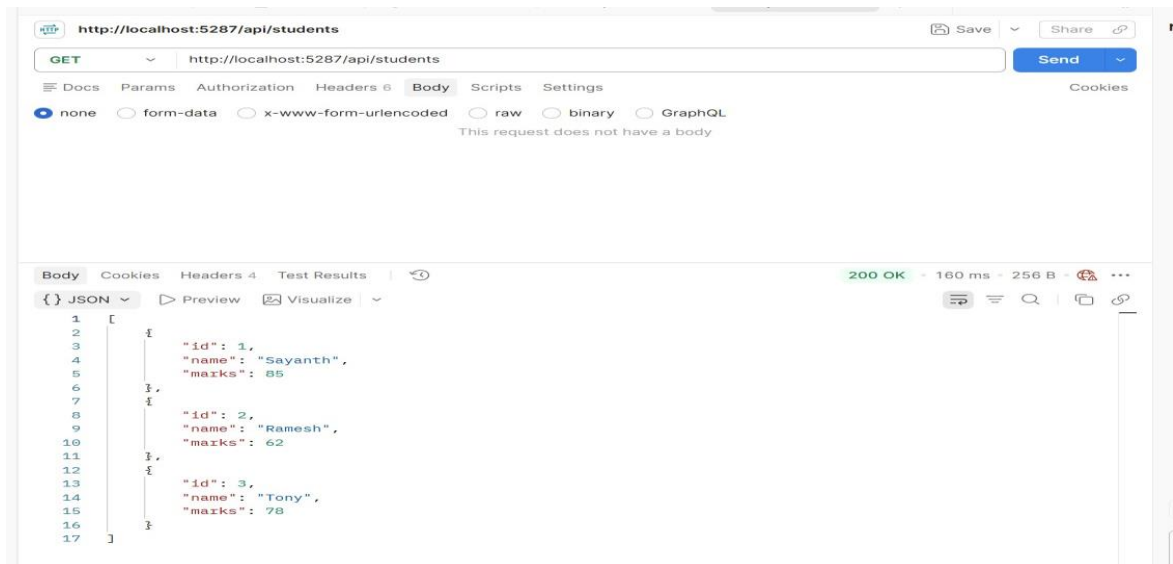
        [HttpGet("{id}")]
        public IActionResult GetStudentById(int id)
    }
}
```

```
{ return Ok(students.First(s => s.Id == id)); }  
[HttpGet("name/{studentName}")] public IActionResult  
GetStudentByName(string studentName)  
{ return Ok(students.First(s => s.Name == studentName)); }  
[HttpGet("marks/{marks:int}")] public  
IActionResult GetStudentMarks(int marks)  
{ return Ok(students.Where(s => s.Marks == marks)); } }
```

Program.cs code:

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllers(); builder.Services.AddOpenApi();  
var app = builder.Build(); if  
(app.Environment.IsDevelopment()) {  
app.MapOpenApi();  
app.UseHttpsRedirection();  
app.UseAuthorization();  
app.MapControllers(); app.Run();
```

Output:



Practical 7:

Aim: Working with Configuration

1. Create custom configuration section in appsettings.json
2. Bind configuration using IOptions
3. Read configuration values inside controller

Codes:

appsettings.json code:

```
{ "CollegeSettings":  
  { "CollegeName": "S. M. Shetty College",  
    "Department": "Information Technology",  
    "Course": "B.Sc. Information Technology",  
    "Semester": "V", "Subject": "ASP.NET Core" },  
    "Logging": { "LogLevel": {  
      "Default": "Information", "Microsoft.AspNetCore": "Warning" } }, "AllowedHosts": "*" }
```

Models/CollegeSettings.cs code:

```
namespace  
Practical7_Configuration.Models { public  
class CollegeSettings { public string  
CollegeName { get; set; } public string  
Department { get; set; } public string Course
```

```
{ get; set; } public string Semester { get; set; }  
}
```

```
public string Subject { get; set; } }
```

Program.cs code:

```
using Practical7_Configuration.Models; var  
builder = WebApplication.CreateBuilder(args);  
builder.Services.Configure<CollegeSettings>(  
builder.Configuration.GetSection("CollegeSettings"));  
builder.Services.AddControllers(); builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
var app = builder.Build(); if  
(app.Environment.IsDevelopment()) { app.UseSwagger();  
app.UseSwaggerUI(); }  
app.UseHttpsRedirection(); app.UseAuthorization();  
app.MapControllers(); app.Run();
```

Controllers/CollegeController.cs code:

```
using Microsoft.AspNetCore.Mvc; using  
Microsoft.Extensions.Options; using  
Practical7_Configuration.Models; namespace  
Practical7_Configuration.Controllers { [ApiController]  
[Route("api/[controller]")] public class CollegeController :  
ControllerBase { private readonly CollegeSettings  
_collegeSettings; public  
CollegeController(IOptions<CollegeSettings> options)
```

```
{ _collegeSettings = options.Value; }

[HttpGet] public IActionResult

GetCollegeSettings() { return Ok(new {

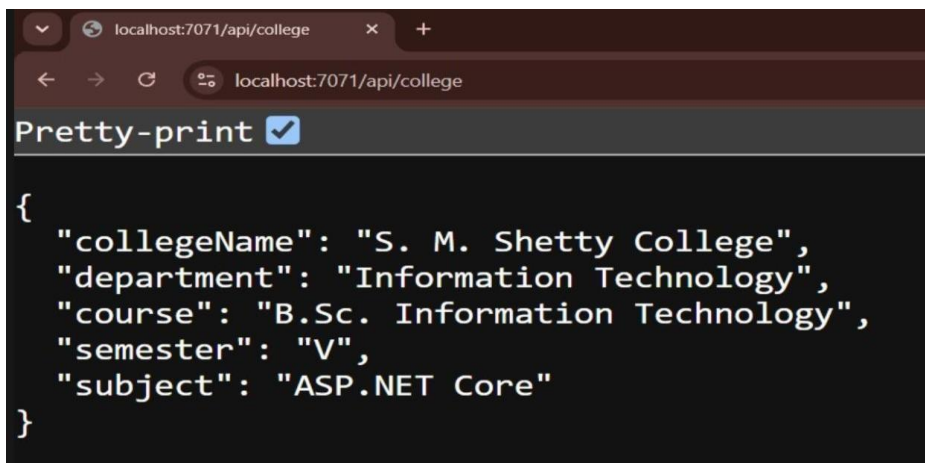
CollegeName                                =

collegeSettings.CollegeName,

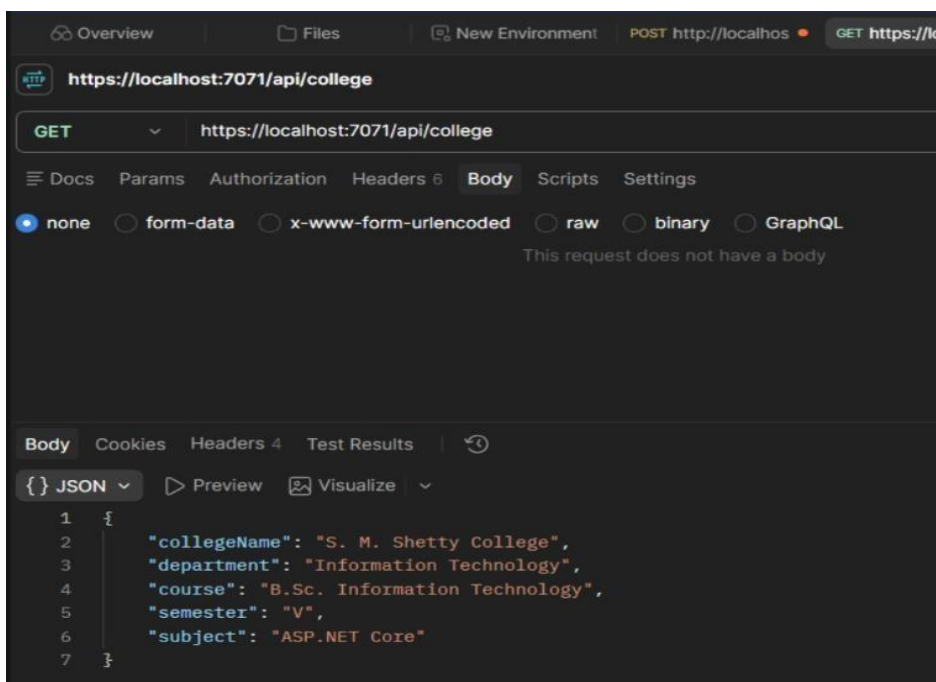
Department = collegeSettings.Department, Course = collegeSettings.Course,

Semester = collegeSettings.Semester, Subject = collegeSettings.Subject }); } }
```

Output:



```
{
  "collegeName": "S. M. Shetty College",
  "department": "Information Technology",
  "course": "B.Sc. Information Technology",
  "semester": "V",
  "subject": "ASP.NET Core"
}
```



```
1 {
2   "collegeName": "S. M. Shetty College",
3   "department": "Information Technology",
4   "course": "B.Sc. Information Technology",
5   "semester": "V",
6   "subject": "ASP.NET Core"
7 }
```

Practical 8:

Aim: Integrating EF Core with SLite

1. Install EF Core packages
2. Create DbContext and Entity
3. Perform Insert and Retrieve operations

Codes:

Models/Product.cs code:

```
namespace Practical8_EFCoreSQLite.Models
{
    public class Product {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
}
```

Data/AppDbContext.cs code:

```
using Microsoft.EntityFrameworkCore;
using Practical8_EFCoreSQLite.Models;
namespace Practical8_EFCoreSQLite.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions
        <AppDbContext> options) : base(options) { }
        public DbSet<Product> Products { get; set; }
    }
}
```

appsettings.json code:

```
{ "ConnectionStrings":  
  { "DefaultConnection": "Data Source=products.db" },  
  "Logging": { "LogLevel":  
    { "Default": "Information", "Microsoft.AspNetCore": "Warning" } },  
  "AllowedHosts": "*" }
```

Program.cs Code:

```
Microsoft.EntityFrameworkCore; using Practical8_EFCoreSQLite.Data; var  
builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllers();  
builder.Services.AddDbContext<AppDbContext>(options =>  
  options.UseSqlite(  
    builder.Configuration.GetConnectionString("DefaultConnection")));  
builder.Services.AddOpenApi(); var app = builder.Build(); if  
(app.Environment.IsDevelopment()) { app.MapOpenApi(); }  
app.UseHttpsRedirection(); app.UseAuthorization();  
app.MapControllers(); app.Run();
```

Controllers/ProductsController.cs code:

```

using Microsoft.AspNetCore.Mvc; using
Practical8_EFCoreSQLite.Data; using
Practical8_EFCoreSQLite.Models; namespace
Practical8_EFCoreSQLite.Controllers { [ApiController]
[Route("api/[controller]")] public class ProductsController :
ControllerBase { private readonly ApplicationDbContext _context; public
ProductsController(ApplicationDbContext context) { _context = context; }
[HttpPost] public IActionResult
AddProduct(Product product) {
_context.Products.Add(product);
_context.SaveChanges(); return Created("api/products/"
+ product.Id, product); }
[HttpGet] public IActionResult GetProducts() { var products =
_context.Products.ToList(); return Ok(products); } } }

```

Output:

ce. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules.

The screenshot shows the Postman interface for a POST request to `https://localhost:7063/api/products`. The request body is a JSON object: `{ "id": 2, "name": "Monitor", "price": 12000 }`. The response is a `201 Created` status with a response time of 154 ms and a body size of 218 B. The response body is a JSON object: `{ "id": 2, "name": "Monitor", "price": 12000 }`.

```
POST https://localhost:7063/api/products
```

```
{
  "id": 2,
  "name": "Monitor",
  "price": 12000
}
```

201 Created • 154 ms • 218 B

```
{
  "id": 2,
  "name": "Monitor",
  "price": 12000
}
```

The screenshot shows the Postman interface for a GET request to `https://localhost:7063/api/products`. The response is a `200 OK` status with a response time of 94 ms and a body size of 228 B. The response body is a JSON array containing two product objects: `[ { "id": 1, "name": "Laptop", "price": 55000 }, { "id": 2, "name": "Monitor", "price": 12000 } ]`.

```
GET https://localhost:7063/api/products
```

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  },
  {
    "id": 2,
    "name": "Monitor",
    "price": 12000
  }
]
```

200 OK • 94 ms • 228 B

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 55000
  },
  {
    "id": 2,
    "name": "Monitor",
    "price": 12000
  }
]
```

Practical 9:

Aim: Applying Migrations

1. Add Migrations
2. Update Database
3. Perform simple LINQ queries

Codes:

Models/Product.cs code:

```
namespace Practical9_MigrationsLINQ.Models
{
    public class Product {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
        public string Category { get; set; }
    }
}
```

Data/AppDbContext.cs code:

```
using Microsoft.EntityFrameworkCore;
using PracticalG_MigrationsLINQ.Models;

namespace PracticalG_MigrationsLINQ.Data
{
    public class AppDbContext : DbContext {
        AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public DbSet<Product> Products { get; set; }
    }
}
```

appsettings.json code:

```
{"ConnectionStrings":  
 { "DefaultConnection": "Data Source=products_linq.db" },  
 "Logging": { "LogLevel":  
 { "Default": "Information", "Microsoft.AspNetCore":  
 "Warning" } }, "AllowedHosts": "*" }
```

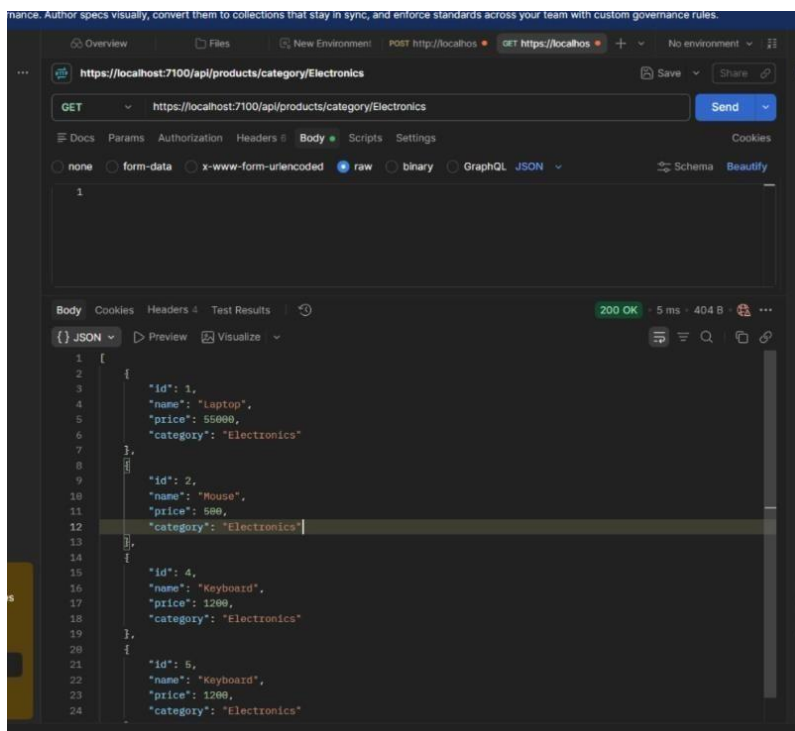
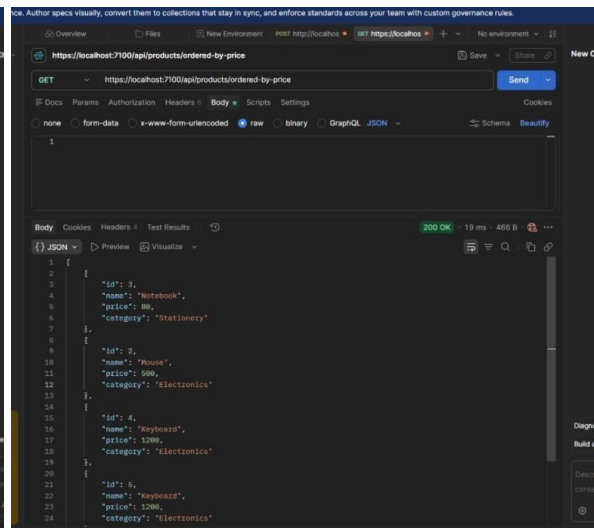
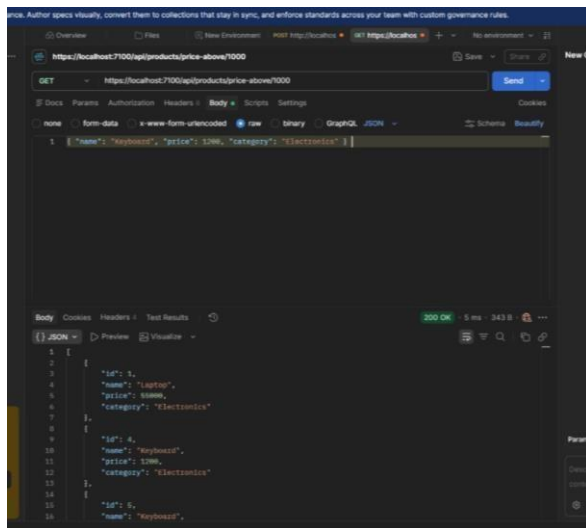
Program.cs code:

```
using Microsoft.EntityFrameworkCore; using PracticalG_MigrationsLINQ.Data;  
var builder =  
WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllers();  
builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlite(  
builder.Configuration.GetConnectionString("DefaultConnection")));  
builder.Services.AddOpenApi(); var app = builder.Build(); if  
(app.Environment.IsDevelopment())  
{ app.MapOpenApi(); } app.UseHttpsRedirection();  
app.UseAuthorization(); app.MapControllers(); app.Run();
```

Controllers/ProductsController.cs code:

```
using Microsoft.AspNetCore.Mvc; using
PracticalG_MigrationsLINQ.Data; using
PracticalG_MigrationsLINQ.Models; namespace
Practical9_MigrationsLINQ.Controllers      {
[ApiController] [Route("api/[controller]")] public
class ProductsController : ControllerBase {
private readonly ApplicationDbContext _context; public
ProductsController(ApplicationDbContext context)
{ _context = context; }
[HttpPost] public IActionResult AddProduct(Product product) {
_context.Products.Add(product); _context.SaveChanges();
return Created("api/products/" + product.Id, product);
}[HttpGet] public IActionResult GetAllProducts() { var products =
_context.Products.ToList(); return Ok(products); }
[HttpGet("price-above/{amount}")] public IActionResult
GetProductsPriceAbove(double amount) { var products =
_context.Products .Where(p => p.Price > amount) .ToList(); return
Ok(products); }
[HttpGet("category/{categoryName}")] public IActionResult
GetProductsByCategory(string categoryName) { var products = _context.Products
.Where(p => p.Category == categoryName) .ToList(); return Ok(products); }
[HttpGet("ordered-by-price")] public IActionResult GetProductsOrderedByPrice() {
var products = _context.Products .OrderBy(p => p.Price) .ToList(); return
Ok(products); } }
```

Output:



Practical 10:

Aim: Implementing Basic Authentication

1. Configure simple identity setup
2. Protect one controller endpoint
3. Test authorized vs unauthorized access

Codes:

Data/ApplicationDbContext.cs code:

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore; namespace
Practical10_BasicAuthentication.Data
{ public class ApplicationDbContext : IdentityDbContext
{ public ApplicationDbContext( DbContextOptions<ApplicationDbContext> options)
: base(options) { } } }
```

appsettings.json code:

```
{ "ConnectionStrings": { "DefaultConnection":
"Data Source=identity_demo.db" },
"Logging": { "LogLevel": { "Default": "Information",
"Microsoft.AspNetCore": "Warning" } },
"AllowedHosts": "*" }
```

Program.cs code:

```
using Microsoft.AspNetCore.Identity; using
Microsoft.EntityFrameworkCore; using
Practical10_BasicAuthentication.Data; var builder =
WebApplication.CreateBuilder(args);
builder.Services.AddControllers(); builder.Services.AddOpenApi();
builder.Services.AddDbContext<ApplicationDbContext>
(options => options.UseSqlite(
builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders(); builder.Services.ConfigureApplicationCookie
(options => { options.Events.OnRedirectToLogin = context =>
{ context.Response.StatusCode = 401; return
Task.CompletedTask; };
options.Events.OnRedirectToAccessDenied = context => { context.Response.StatusCode =
403; return Task.CompletedTask; }; });
var app = builder.Build(); if
(app.Environment.IsDevelopment())
{ app.MapOpenApi(); } app.UseHttpsRedirection();
app.UseAuthorization(); app.MapControllers(); app.Run();
```

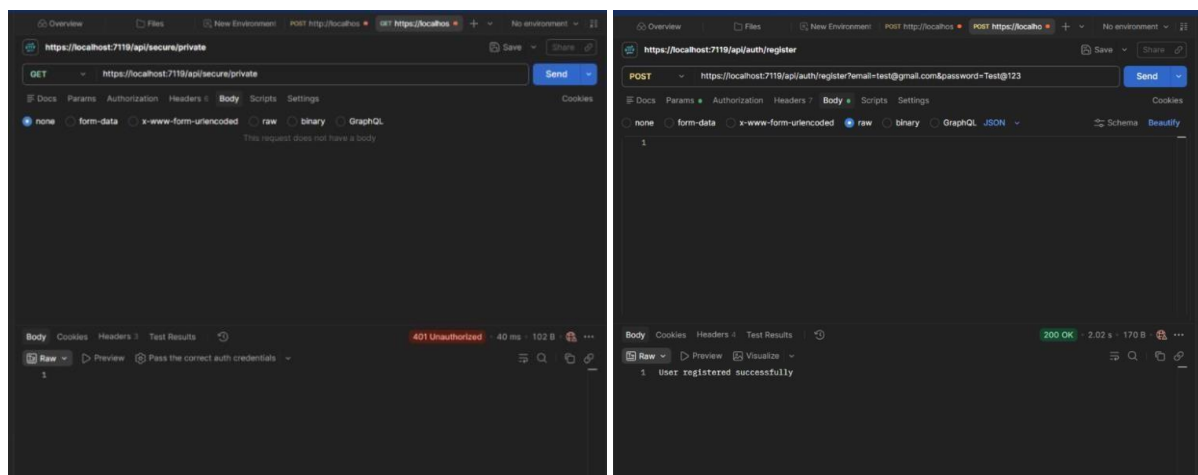
Controllers/AuthController.cs code:

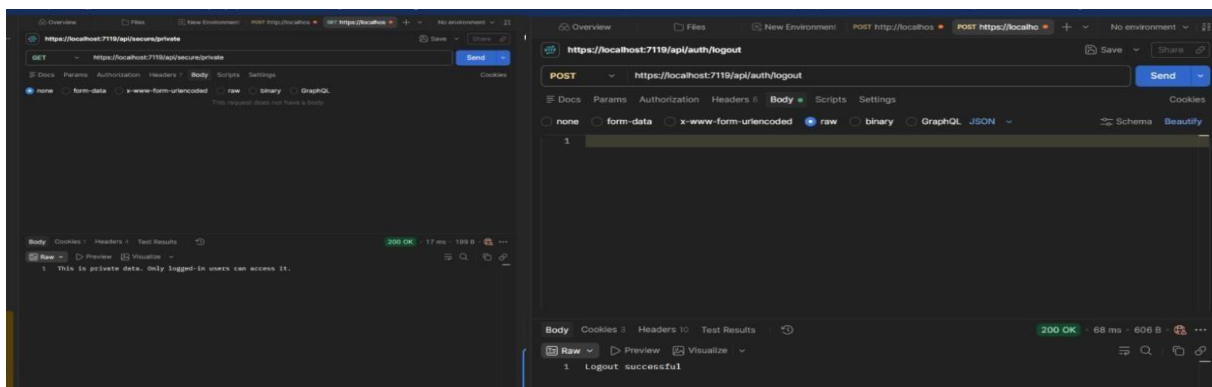
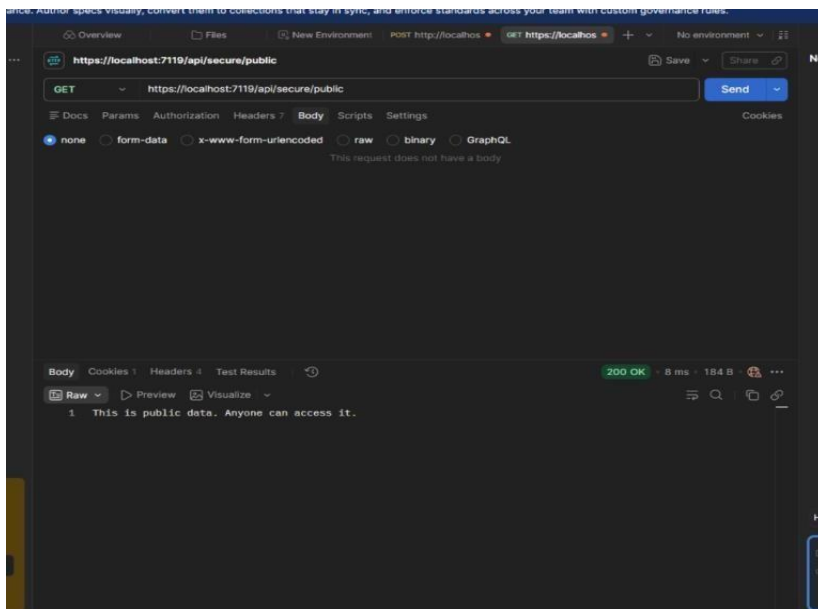
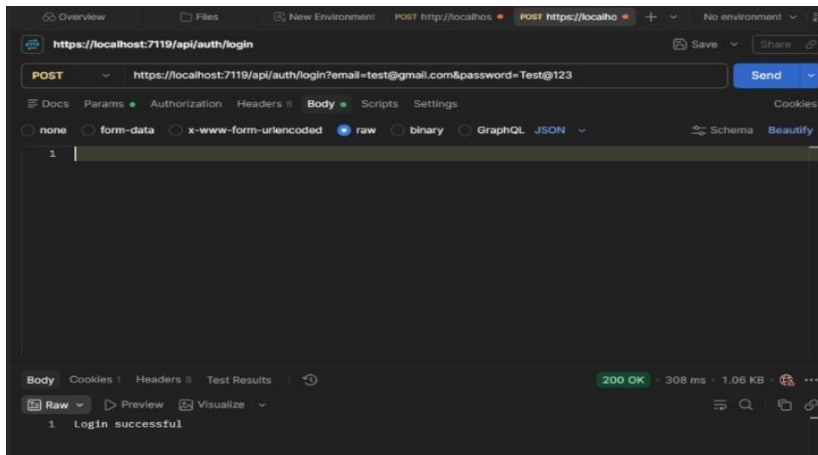
```
using Microsoft.AspNetCore.Identity; using
Microsoft.AspNetCore.Mvc; namespace
Practical10_BasicAuthentication.Controllers {
[ApiController] [Route("api/[controller]")] public
class AuthController : ControllerBase
{ private readonly UserManager<IdentityUser> _userManager; private
readonly SignInManager<IdentityUser> _signInManager;
public AuthController( UserManager<IdentityUser> userManager,
SignInManager<IdentityUser> signInManager)
{ } _userManager = userManager; _signInManager = signInManager;
[HttpPost("register")] public async Task<IActionResult> Register(string
email, string password) { var user = new IdentityUser { Username =
email, Email = email }; var result = await
_userManager.CreateAsync(user, password); if (result.Succeeded) {
return Ok("User registered successfully"); } return
BadRequest(result.Errors); }
[HttpPost("login")] public async Task<IActionResult> Login(string email, string
password) { var result = await _signInManager.PasswordSignInAsync( email,
password, false, false); if (result.Succeeded) { return Ok("Login successful"); } return
Unauthorized("Invalid login details"); }
[HttpPost("logout")] public async Task<IActionResult> Logout()
{ await _signInManager.SignOutAsync(); return
Ok("Logout successful"); } }
```

Controllers/SecureController.cs code:

```
using Microsoft.AspNetCore.Authorization; using
Microsoft.AspNetCore.Mvc; namespace
Practical10_BasicAuthentication.Controllers {
[ApiController] [Route("api/[controller]")] public
class SecureController : ControllerBase {
[HttpGet("public")] public IActionResult PublicData()
{ return Ok("This is public data. Anyone can access it."); } [Authorize]
[HttpGet("private")] public
ActionResult PrivateData()
{ return Ok("This is private data. Only logged-in users can access it."); } }
```

Output:





Practical 11:

Aim: Implementing Logging

1. Use ILogger in controller
2. Configure log levels in appsettings.json
3. Log information and error messages

Codes:

appsettings.json:

```
{ "Logging": { "LogLevel":  
  { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }  
  , "AllowedHosts": "*" }
```

Controllers/LoggingController.cs code:

```
using Microsoft.AspNetCore.Mvc; namespace  
Practical11_Logging.Controllers { [ApiController]  
[Route("api/[controller]")] public class LoggingController :  
ControllerBase { private readonly  
ILogger<LoggingController> _logger; public  
LoggingController( ILogger<LoggingController> logger)  
{ } _logger = logger; [HttpGet("info")] public  
ActionResult InformationLog()  
{ _logger.LogInformation( "Information log generated successfully."); return  
Ok("Information log created."); }
```

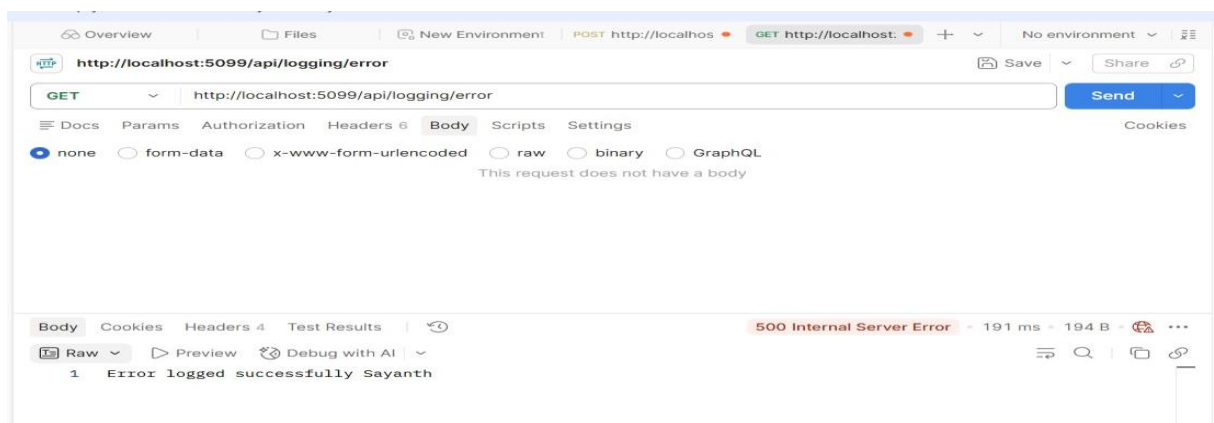
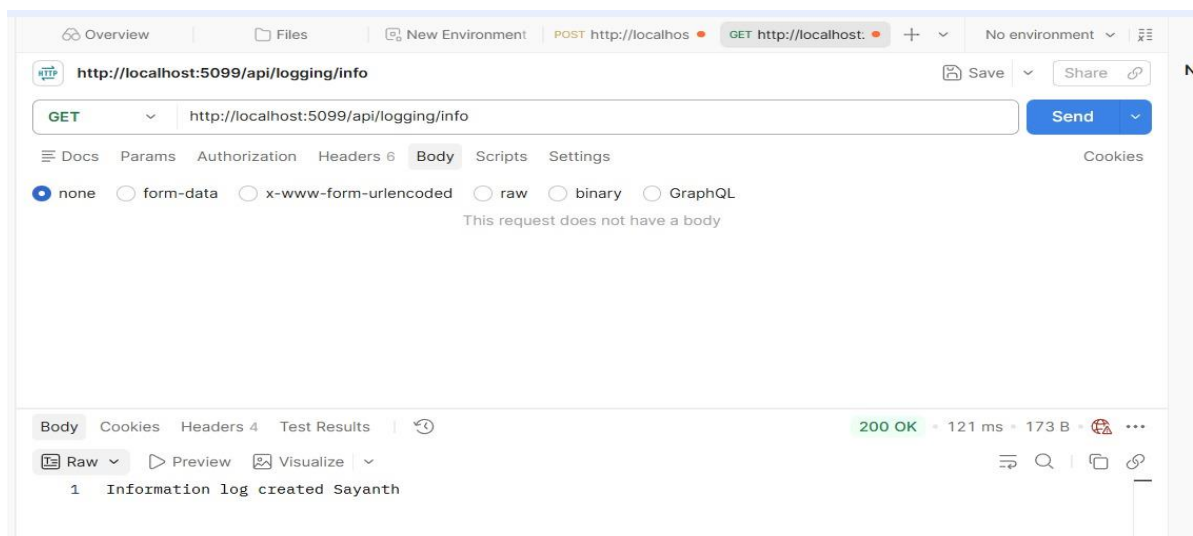
```

[HttpGet("student")] public
ActionResult StudentLog()
{
    _logger.LogInformation( "Student endpoint accessed."); return
    Ok("Student endpoint executed."); }

[HttpGet("error")] public
ActionResult ErrorLog()
{
    try { int number = 10; int result = number / 0; return
    Ok(result); } catch (Exception ex)
    {
        _logger.LogError( ex, "An error occurred while processing request."); return
        StatusCode( 500, "Error logged successfully."); } } }

```

Output:



Practical 12:

Aim: Implementing Basic Caching

1. Use IMemoryCache
2. Cache GET request results
3. Demonstrate cached

Codes:

Program.cs code:

```
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllers();  
builder.Services.AddMemoryCache();  
builder.Services.AddOpenApi(); var app =  
builder.Build(); if  
(app.Environment.IsDevelopment()) {  
app.MapOpenApi(); } app.UseHttpsRedirection();  
app.UseAuthorization(); app.MapControllers();  
app.Run();
```

Controllers/CacheController.cs code:

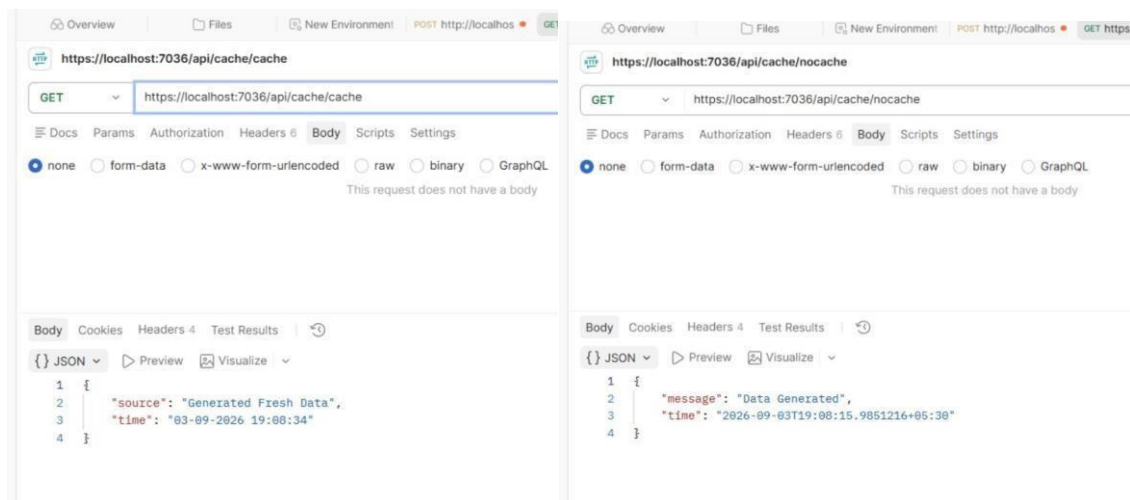
```
using Microsoft.AspNetCore.Mvc; using  
Microsoft.Extensions.Caching.Memory;  
namespace Practical12_Caching.Controllers
```

```

{ [ApiController] [Route("api/[controller]")]
public class CacheController :
ControllerBase { private readonly
IMemoryCache _cache; public
CacheController(IMemoryCache cache) {
_cache = cache; } [HttpGet("nocache")]
public IActionResult GetWithoutCache()
{ return Ok(new { Message = "Data Generated", Time = DateTime.Now }); }
[HttpGet("cache")] public IActionResult GetWithCache() { string
cacheKey = "CurrentTime"; if (!_cache.TryGetValue(cacheKey, out
string cachedTime))
{ cachedTime = DateTime.Now.ToString(); var cacheOptions = new
MemoryCacheEntryOptions() .SetAbsoluteExpiration(
TimeSpan.FromSeconds(30)); _cache.Set( cacheKey, cachedTime,
cacheOptions); return Ok(new { Source = "Generated Fresh Data", Time =
cachedTime }); } return Ok(new { Source = "Returned From Cache", Time =
cachedTime }); } } }

```

Output:



Practical 13:

Aim: Creating Razor Pages Application

1. Create Razor Page
2. Implement OnGet and OnPost
3. Create simple form

Codes:

Pages/Student.cshtml code:

```
@page
@model Practical13_RazorPages.Pages.StudentModel
@{
    ViewData["Title"] = "Student Form";
}
<h2>Student Information Form</h2>
<form method="post">
    <table>
        <tr>
            <td>Student Name:</td>
            <td>
                <input asp-for="StudentData.Name" />
            </td>
        </tr>
        <tr>
            <td>Course:</td>
            <td>
                <input asp-for="StudentData.Course" />
            </td>
        </tr>
    </table>
</form>
```

```
</tr>
<tr>
    <td>Semester:</td>
    <td>
        <input asp-for="StudentData.Semester" />
    </td>
</tr>
<tr>
    <td colspan="2">
        <button type="submit">Submit</button>
    </td>
</tr>
</table>
</form>
<hr />
<h3>@Model.Message</h3>
@if (Model.IsSubmitted)
{
    <h3>Submitted Student Details</h3>
    <p><b>Name:</b> @Model.StudentData.Name</p>
    <p><b>Course:</b> @Model.StudentData.Course</p>
    <p><b>Semester:</b> @Model.StudentData.Semester</p>
}
```

Pages/Student.cshtml.cs code:

```
using Microsoft.AspNetCore.Mvc; using
Microsoft.AspNetCore.Mvc.RazorPages;
namespace
Practical13_RazorPages.Pages
{ public class StudentModel : PageModel { [BindProperty]
public StudentInfo StudentData { get; set; } = new
StudentInfo(); public string Message { get; set; } = "";
public bool IsSubmitted { get; set; } = false; public void
OnGet() { Message = "Please enter student details."; }
public void
OnPost() { Message = "Form submitted successfully.";
IsSubmitted = true; } } public class StudentInfo
{ public string Name { get; set; } = ""; public
string Course { get; set; } = ""; public string
Semester { get; set; } = ""; } }
```

Output:

The screenshot displays a web application interface. At the top, there is a navigation bar with the text 'Practical13_RazorPages' followed by links for 'Home' and 'Privacy'. Below this, the main heading is 'Student Information Form'. The form contains three input fields: 'Student Name' with the value 'Sayanth', 'Course' with the value 'TYIT', and 'Semester' with the value '5'. A 'Submit' button is located below these fields. Below the form, a message states 'Form submitted successfully. Submitted Student Details'. This message is followed by the submitted data: 'Name: Sayanth', 'Course: TYIT', and 'Semester: 5'. At the bottom of the page, a footer contains the copyright notice '© 2026 - Practical13_RazorPages - Privacy' with a link to the privacy policy.

Practical13_RazorPages Home Privacy

Student Information Form

Student Name:

Course:

Semester:

Form submitted successfully.
Submitted Student Details

Name: Sayanth

Course: TYIT

Semester: 5

© 2026 - Practical13_RazorPages - [Privacy](#)

Practical 14:

Aim: Form Validation and Data Display

1. Apply model Validation
2. Display validation messages
3. Show data in Razor Pages

Codes:

Pages/StudentValidation.cshtml code:

```
@page
@model Practical14_FormValidation.Pages.StudentValidationModel
@{
    ViewData["Title"] = "Student Form Validation";
}
<h2>Student Registration Form</h2>
<form method="post">
    <div asp-validation-summary="ModelOnly" style="color:red"></div>
    <table>
        <tr>
            <td>Student Name:</td>
            <td>
                <input asp-for="StudentData.Name" />
                <br />
                <span asp-validation-for="StudentData.Name" style="color:red"></span>
            </td>
        </tr>
        <tr>
            <td>Email:</td>
```

```

<td>

    <input asp-for="StudentData.Email" />

    <br />

    <span asp-validation-for="StudentData.Email" style="color:red"></span>

</td>
</tr>
<tr>
    <td>Age:</td>
    <td>

        <input asp-for="StudentData.Age" type="number" />

        <br />

        <span asp-validation-for="StudentData.Age" style="color:red"></span>

    </td>
</tr>
<tr>
    <td>Course:</td>
    <td>

        <select asp-for="StudentData.Course">
            <option value="">-- Select Course --</option>
            <option value="B.Sc. IT">B.Sc. IT</option>
            <option value="B.Sc. CS">B.Sc. CS</option>
            <option value="B.Sc. DS">B.Sc. DS</option>
        </select>

        <br />

        <span asp-validation-for="StudentData.Course" style="color:red"></span>

    </td>
</tr>
<tr>
    <td colspan="2">

```

```

        <br />
        <button type="submit">Submit</button>
    </td>
</tr>
</table>
</form>
<hr />
<h3>@Model.Message</h3>
@if (Model.IsSubmitted)
{
    <h3>Submitted Student Details</h3>
    <table border="1" cellpadding="8">
        <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Age</th>
            <th>Course</th>
        </tr>
        <tr>
            <td>@Model.StudentData.Name</td>
            <td>@Model.StudentData.Email</td>
            <td>@Model.StudentData.Age</td>
            <td>@Model.StudentData.Course</td>
        </tr>
    </table>
}
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Pages/StudentValidation.cshtml.cs code:

```
using Microsoft.AspNetCore.Mvc; using
Microsoft.AspNetCore.Mvc.RazorPages; using
System.ComponentModel.DataAnnotations;
namespace Practical14_FormValidation.Pages
{ public class StudentValidationModel : PageModel
{ [BindProperty] public StudentInfo StudentData { get; set; } = new StudentInfo();
public string Message { get; set; } = ""; public bool IsSubmitted { get; set; } =
false; public void OnGet() { Message = "Please fill the student form."; } public
IActionResult OnPost() { if (!ModelState.IsValid)
{ Message = "Please correct the validation errors."; IsSubmitted = false;
return Page(); } Message = "Form submitted successfully."; IsSubmitted = true;
return Page(); } } public class StudentInfo
{ [Required(ErrorMessage = "Student name is required")]
[StringLength(30, ErrorMessage = "Name cannot be more than 30 characters")]
public string Name { get; set; } = "";
[Required(ErrorMessage = "Email is required")]
[EmailAddress(ErrorMessage = "Enter valid email address")] public string Email {
get; set; } = ""; [Required(ErrorMessage = "Age is required")]
[Range(18, 30, ErrorMessage = "Age must be between 18 and 30")] public
int Age { get; set; } [Required(ErrorMessage = "Please select course")]
public string Course { get; set; } = ""; } }
```

Output:

Student Registration Form

Student Name:
Student name is required

Email:
Email is required

Age:
Age must be between 18 and 30

Course:
Please select course

Please fill the student form.

Practical 15:

Aim: Writing Basic Unit Tests

1. Create xUnit test project
2. Write test for service method
3. Execute tests using CLI

Codes:

StudentService.cs code:

```
namespace Practical15_Services { public class StudentService { public string GetResult(int marks) { if (marks >= 40) { return "Pass"; } else { return "Fail"; } } }
```

UnitTest1.cs

```
using Practical15_Services; using
Xunit; namespace Practical15_Tests {
public class UnitTest1 { [Fact]
public void GetResult_MarksGreaterThan40_ReturnsPass()
{ // Arrange StudentService service = new StudentService();
// Act string result = service.GetResult(75); // Assert
Assert.Equal("Pass", result); }
[Fact] public void GetResult_MarksLessThan40_ReturnsFail()
{ // Arrange StudentService service = new StudentService(); // Act string result =
service.GetResult(25); //
Assert Assert.Equal("Fail", result); } }
```

Output:

The screenshot displays the Test Explorer interface in an IDE. The main pane shows a tree view of test results. The test run is completed, showing 2 tests passed, 0 failed, and 0 skipped, with a total duration of 236 ms. The test results are as follows:

Test	Duration	Traits	Error Message
Practical15_Tests (2)	5 ms		
Practical15_Tests (2)	5 ms		
UnitTest1 (2)	5 ms		
GetResult_MarksGreaterThan40...	< 1 ms		
GetResult_MarksLessThan40_R...	5 ms		

The right-hand pane shows the Group Summary for Practical15_Tests, indicating that 2 tests in the group passed with a total duration of 5 ms. The Outcomes section shows 2 Passed.